Модульное программирование

Стандартные модули еще

Что это

Модульное программирование — это подход к разработке программ, который основывается на разделении программы на независимые, взаимосвязанные блоки, называемые **модулями**.

Каждый модуль решает определённую задачу и может быть разработан, протестирован и отлажен отдельно от других модулей.

Что это

Модуль — это часть программы, которая может быть использована независимо от других частей программы. Модуль включает в себя функции, переменные и типы данных, которые могут быть скрыты (private) или доступны для использования другими модулями (public).

Преимущества модульного подхода

- Ускорение компиляции и сборки
- Борьба со сложностью и читаемость кода
- Повторное использование
- Изоляция и безопасность
- Простота тестирования и отладки
- Параллельная разработка

Старый подход

В C++ модулем может считаться файл заголовка (header) и файл реализации:

1. Заголовочный файл (*.h) — содержит объявления (прототипы функций, классов и переменных), которые могут быть использованы другими модулями.

2. Файл реализации (*.cpp) — содержит определения функций, классов и других элементов программы.

Интерфейс модуля

Описывается в заголовочном файле (*.h).

- Содержит объявления функций, классов и переменных, которые могут быть использованы другими модулями.

```
// math_operations.h
#ifndef MATH_OPERATIONS_H
#define MATH_OPERATIONS_H


int add(int a, int b);
int subtract(int a, int b);


#endif // MATH_OPERATIONS_H
```

Интерфейс модуля

Описывается в заголовочном файле (*.h).

- Конструкция #ifndef MATH_OPERATIONS_H ... используется для контроля за тем, чтобы конкретный исходный файл при компиляции в объектный файл подключался строго один раз, для тех же целей можно использовать директиву #pragma once:

```
// math_operations.h
#pragma once

int add(int a, int b);
int subtract(int a, int b);
```

Реализация модуля

Описывается в файле исходного кода (*.cpp).

- Содержит реализацию функций и классов, объявленных в заголовочном файле.

```
// math_operations.cpp
#include "math_operations.h"

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}
```

Компиляция и компоновка программы

Компиляция (Compilation) — процесс перевода исходного кода из языка высокого уровня (например, C++) в машинный код или промежуточный объектный код (object files).

Компоновка (Linking) — процесс объединения всех скомпилированных объектных файлов и библиотек в один исполняемый файл. На этом этапе программа собирается в единое целое, и разрешаются все внешние ссылки (например, вызовы функций, реализованных в других модулях).

Пример программы с модулями

```
// main.cpp
#include <iostream>
#include «math_operations.h" //подключение модуля


int main() {
    int a = 10, b = 5;


    std::cout << "Add: " << add(a, b) << std::endl; //вызов функции Add из модуля
    std::cout << "Subtract: " << subtract(a, b) << std::endl; //вызов функции subtract из модуля


    return 0;
}
```

Пример программы с модулями

Этот пример программы включает два модуля:

- math_operations.h и math_operations.cpp — модуль с математическими операциями.
- main.cpp — основной модуль, который использует функции модуля math_operations.

Современный подход

Начиная с C++20, вместо текстового включения #include используется ключевое слово import. Модули компилируются один раз в бинарный формат, что кратно ускоряет сборку проекта и полностью изолирует макросы

Файлы интерфейса модулей обычно имеют расширения .ixx (в MSVC) или .cppm (в Clang/GCC).

Пример 2

Math.ixx

```
export module Math; // Объявляем имя модуля


export int add(int a, int b) { // Ключевое слово export делает функцию доступной снаружи
    return a + b;
}


int internal_helper() { // Без export функция останется скрытой внутри модуля
    return 42;
}
```

Пример 2

main.cpp

```
import Math;  // Импортируем наш модуль
import std;   // В C++23 можно импортировать всю стандартную библиотеку


int main() {
    int result = add(5, 7);
    // internal_helper(); // Ошибка компиляции: функция изолирована
    return 0;
}
```